SYSTEMS PROPOSAL

Product Requirements Document

Automated Library System

A Web-Based Library Management System for the KNUST Libraries

Project Proposal | Group 2 | KNUST

1. Problem Identification

The library system of the Kwame Nkrumah University of Science and Technology (KNUST), anchored by the Prempeh II Library in Kumasi, serves a university community numbering in the tens of thousands. Almost all of the work that keeps that library running, issuing a book, taking it back, working out whether a book is late and by how much, deciding who gets a title that everyone wants, is still carried out by hand on paper.

The first cost falls on students. A student who needs a prescribed text has no way of knowing, before walking to the library, whether a copy is on the shelf. If every copy is out, there is nothing to do but come back tomorrow and ask again. High demand titles circulate informally: the returned copy goes to whoever happens to be standing at the desk when it arrives, not to whoever asked for it first. Without a queue there is no fairness, only proximity.

The second cost falls on the staff. At the start of a semester the circulation desk becomes a bottleneck, because every loan is written out longhand. Due dates are counted on a calendar. Overdue books are noticed rather than detected, and only when somebody thinks to look. Fines are calculated by hand, and anything calculated by hand is calculated inconsistently.

A paper process creates four downstream failures:

- **Access failure:** students cannot see availability before travelling, cannot reserve a title that is currently out, and cannot find out where they stand in the queue, because there is no queue to stand in.
- **Enforcement failure:** overdue loans are discovered late or not at all. Borrowing limits and fine thresholds exist on paper but cannot be checked at the moment of issue, which is the only moment they matter.
- **Accountability failure:** a fine can be reduced or written off with no durable record of who approved it. When money moves and nobody is named, the library cannot answer a question it may one day be asked.
- **Planning failure:** acquisition budgets are spent without borrowing evidence. Nobody can say which titles are genuinely in demand, which sit unread, or how stock is distributed across subjects.

Existing library management software does not close this gap cleanly. Koha is free, open source and genuinely capable, but it carries a large configuration surface and an interface designed for trained cataloguers rather than for a student on a phone. Commercial products such as Ex Libris Alma and Sierra are priced for well funded institutions and are sold as long term subscriptions. None of them models a Ghanaian university's membership categories, its student and staff identifier formats, or the cedi denominated fine policy out of the box. The practical result is that the work stays on paper.

1.1 Proposed Solution

The Automated Library System (ALS) is a web based library management system built as a single application with two faces: a member portal for students, faculty and public members, and

a back office for librarians and administrators. Both are served by one REST API, so a rule written once is enforced everywhere.

Members get a catalogue they can search by title, author or ISBN from any browser, with live availability shown per title. If every copy is out, a member reserves the title and joins an ordered queue. When a copy comes back, the system promotes the person at the front of that queue automatically, holds the copy for them, and notifies them. The hold expires after a configurable window and passes to the next person, so a queue cannot be blocked by someone who never collects.

Librarians get a circulation console built around a barcode scanner. Checking a book out is two scans: the member, then the copy. Taking a book back is one scan, because the accession number identifies the loan on its own. The console refuses a checkout the member is not entitled to, showing the reason, and it will not release a held copy to anyone other than the member it is being held for. Due dates, overdue status and fines are computed by the system from settings an administrator controls, never by hand.

Everything that moves stock, moves money, or changes an account is recorded against the person who did it. Scheduled jobs run in the background: a nightly sweep marks overdue loans and notifies the borrowers, a daily job reminds members of loans about to fall due, and an hourly job expires uncollected holds. Notifications appear in the application and are dispatched by email. Administrators get a dashboard of live figures, borrowing trends, stock health and an audit trail.

2. Actors

ALS has three in-application actors arranged as a strict hierarchy: Member, Librarian and Administrator, in ascending order of authority. Every higher role inherits the permissions of the roles beneath it. Everyone else, and every other system, sits outside the application and interacts only with what it sends out or what it reads in.

2.1 In-App Users

These are the only actors who hold an account and sign in.

2.1.1 Primary Actor: Library Member (Student, Faculty or Public)

Attribute	Details
Role	The borrower. Searches the catalogue, reserves titles, tracks their own loans, renews them, and settles fines.
Demographics	Undergraduate and postgraduate students, teaching and research staff, and members of the public with borrowing rights. Predominantly mobile browsers on intermittent campus connectivity.
Goals	To find out whether a book is available without walking to the library, to secure a place in the queue for a title in demand, and to avoid a fine by being told a book is due before it is late.

Attribute	Details
Frustrations	Travels to the library to discover every copy is out. Has no way to claim a title in advance. Discovers a fine only when returning the book, with no way to check the amount beforehand.
Key Interactions	Registers with name, email, password and phone number. Signs in with either the email address or the member ID. Searches and filters the catalogue, opens a book to see live availability, reserves a title, views the loan list with due dates, renews within the allowance, reads notifications, and views fines and the outstanding total.
Access Level	MEMBER. Own records only. Cannot see another member's loans, fines or details.
Quote	"I walked to the library three times for the same textbook. Nobody could tell me when it was coming back."

2.1.2 Secondary Actor: Librarian (Circulation Desk Staff)

Attribute	Details
Role	Runs the desk. Issues and receives books, manages the catalogue and physical stock, works the reservation queue, and settles fines.
Demographics	Library assistants and professional librarians working a shift at a service point, usually on a desktop with a handheld barcode scanner.
Goals	To clear a queue of students quickly without transcription errors, to be told at the moment of issue whether a member is entitled to borrow, and to have due dates and fines computed rather than worked out.
Frustrations	Manual issue is slow when a queue forms. Borrowing limits and unpaid fines cannot be checked at the counter. A returned copy that somebody is waiting for gets shelved by mistake.
Key Interactions	Scans a member and a copy to check out, scans a copy to take it back, adds and edits catalogue records, adds copies in batches with sequential accession numbers, updates copy status, views the reservation queue, records fine payments and waivers, and suspends or reactivates a patron account.
Access Level	LIBRARIAN. Full access to patron records and circulation. Cannot change roles, edit settings, administer another librarian, or read staff oversight reports.
Quote	"I need to know before I hand the book over, not after the student has left the desk."

2.1.3 Tertiary Actor: System Administrator

Attribute	Details
Role	Owns the configuration and the oversight. Sets library policy, manages staff accounts, and reads the audit trail.
Demographics	The head librarian or a systems librarian responsible for the service as a whole rather than for a shift at the desk.

Attribute	Details
Goals	To set loan periods, fine rates, renewal allowances and thresholds without a developer. To see whether the desk is evenly covered. To be able to answer who processed a given transaction.
Frustrations	Policy changes require someone to remember them rather than being enforced. There is no way to trace a written off fine back to the person who approved it.
Key Interactions	Edits system settings, promotes or demotes staff, changes membership categories, administers librarian accounts, reads the per-person desk activity summary, and reads the audit trail.
Access Level	ADMIN. Everything a librarian can do, plus roles, membership, settings, staff activity and the audit log.
Quote	"If a fine gets waived, I need to be able to say who waived it and when."

Design note on privilege. Gating an endpoint by role is not sufficient on its own, because it says nothing about who the target of the action is. A librarian passing a LIBRARIAN check could otherwise suspend an administrator. ALS therefore applies a second test on every account action: an actor may never act on someone who outranks them, may never act on a peer unless the actor is an administrator, and may never act on their own account, which would otherwise allow self-lockout. This rule lives on the server and is covered by tests. The interface additionally hides buttons the server would refuse, so staff are not offered actions that would fail.

2.2 External Systems and Recipients (Not In the App)

These parties and systems have no accounts and no interface inside ALS. They are listed because the systems analysis method requires identifying everything that touches the system's inputs and outputs.

External Party or System	What It Is	How It Interacts with ALS
Member email inbox	Any mail provider used by a student, staff member or public borrower.	Receives welcome messages, due-soon reminders, overdue notices, fine notices, and reservation ready and expiry alerts, dispatched over SMTP. Delivery is one-way, no reply is processed.
Open Library API	A free public bibliographic service operated by the Internet Archive.	Queried by ISBN when staff add a book, to pre-fill title, authors, publisher and year, and to supply cover images for the catalogue. The system remains fully usable when the service is unreachable, staff simply type the fields.
Barcode scanner	A handheld keyboard-wedge scanner at the circulation desk.	Types an accession number into the focused field and presses Enter. ALS keeps focus on the correct field for the current step so a scan is never lost, and returns focus after each scan is processed.

External Party or System	What It Is	How It Interacts with ALS
SMTP relay	The institutional or third-party mail relay configured for the deployment.	Accepts outbound notification mail. In development the transport falls back to a console or Ethereal preview so that no real mail is sent.
System scheduler	Time-based job runner inside the API process.	Triggers the nightly overdue sweep, the daily due-soon reminders and the hourly hold expiry. Can be disabled by configuration for test and staging deployments.

2.3 Actor Interaction Summary

Actor	In the App?	Access Level	Key Actions
Library Member	Yes	MEMBER, own records only	Search catalogue, reserve titles, view and renew loans, read notifications, view own fines
Librarian	Yes	LIBRARIAN, all patron records	Check out, take back, manage catalogue and copies, work the reservation queue, record fine payments and waivers, suspend a patron
Administrator	Yes	ADMIN, full access	Everything a librarian can do, plus roles, membership, settings, staff activity summary and audit trail
Member email inbox	No	None	Receives reminders, overdue notices, fine notices and reservation alerts
Open Library API	No	None	Supplies ISBN metadata and cover images on request, optional
Barcode scanner	No	None	Enters accession numbers at the circulation desk
System scheduler	No	Internal	Runs the overdue sweep, due-soon reminders and hold expiry

Design principle. Every rule that protects the library is enforced on the server, inside a database transaction where money or stock is involved. The browser is treated as a convenience, never as a safeguard. A member who bypasses the interface entirely and calls the API directly is subject to exactly the same borrowing limits, fine thresholds, suspension checks and role restrictions as a member using the screens.

3. Requirements

3.1 Functional Requirements

3.1.1 Authentication and Access Control

ID	Requirement	Priority	Acceptance Criteria
FR01	The system MUST allow a person to register with full name, email address, password and phone number.	P0, Must Have	Given valid details, when registration is submitted, then an account is created with the MEMBER role, the password is stored only as a bcrypt hash at cost 10 or higher, and a welcome notification is issued.
FR02	The system MUST allow sign-in with either the email address or the member ID (student number, index number or staff ID).	P0, Must Have	Given a registered account, when either credential is supplied with the correct password, then an access token and a refresh token are returned.
FR03	The system MUST issue short-lived access tokens and longer-lived refresh tokens, and MUST allow an access token to be renewed without re-entering the password.	P0, Must Have	Given a valid refresh token, when it is presented to the refresh endpoint, then a new access token is returned; given an expired or tampered token, then the request is rejected with 401.
FR04	The system MUST enforce the role hierarchy MEMBER below LIBRARIAN below ADMIN on the server for every protected route.	P0, Must Have	Given a member's token, when a staff-only route is called, then the response is 403 regardless of what the browser displays.
FR05	The system MUST prevent an actor administering an account of equal or higher rank, or their own account.	P0, Must Have	Given a librarian, when they attempt to suspend an administrator, a fellow librarian, or themselves, then the response is 403 and no change is written.
FR06	The system MUST allow a signed-in user to update their own profile and change their own password.	P1, Should Have	Given the current password, when a new password is submitted, then the stored hash changes and the old password no longer authenticates.

3.1.2 Catalogue and Inventory Management

ID	Requirement	Priority	Acceptance Criteria
FR07	The system MUST allow staff to create, update and remove book records carrying ISBN, title, description, publication year, edition, language, category, publisher and one or more authors.	P0, Must Have	Given a completed book form, when it is saved, then the title appears in the catalogue with all supplied fields and its authors linked.
FR08	The system MUST reject a duplicate ISBN.	P0, Must Have	Given an ISBN already held, when a second book is saved with it, then the response is 409 and no record is created.
FR09	The system SHOULD pre-fill a new book record from its ISBN using an external bibliographic service.	P2, Nice to Have	Given a valid ISBN, when lookup is requested, then title, authors, publisher and year are populated for the librarian to confirm or correct; when the service is unavailable, the form stays usable and is filled by hand.

ID	Requirement	Priority	Acceptance Criteria
FR10	The system MUST offer the subject category as a selection from the maintained taxonomy and the publication year as a bounded selection, not as free text.	P1, Should Have	Given the book form, when the category and year fields are opened, then the seeded subject list and a valid year range are presented, and a typed category cannot introduce a duplicate subject.
FR11	The system MUST record each physical copy separately with a unique accession number, a shelf location and a status.	P0, Must Have	Given a book, when a copy is added, then it is listed against that title with its own accession number, and a duplicate accession number is refused.
FR12	The system SHOULD allow several copies to be created in one action with sequential accession numbers.	P1, Should Have	Given a starting accession number and a quantity, when the form is submitted, then that many copies are created with consecutive numbers preserving any zero padding, and if any number in the range is already taken none of the copies are created.
FR13	The system MUST track copy status as one of AVAILABLE, CHECKED_OUT, RESERVED, LOST or DAMAGED, and MUST allow staff to set it.	P0, Must Have	Given a copy marked LOST or DAMAGED, when a checkout is attempted against it, then the checkout is refused.
FR14	The system MUST allow a copy to be located by its accession number so a scanner can drive the desk.	P0, Must Have	Given a scanned accession number, when lookup runs, then the copy, its title, its shelf location and its current status are returned in a single response.

3.1.3 Search and Discovery

ID	Requirement	Priority	Acceptance Criteria
FR15	The system MUST allow the catalogue to be searched by title, by author and by ISBN.	P0, Must Have	Given a search term matching any of the three, when the search runs, then matching titles are returned; an ISBN search returns the exact title.
FR16	The system MUST allow results to be filtered by subject category and by availability, and MUST paginate them.	P0, Must Have	Given a filter selection, when results are returned, then only matching titles appear, with a total count and page controls.
FR17	The system MUST show live availability on the book detail view, distinguishing copies on the shelf from copies out on loan or held.	P0, Must Have	Given a title with copies in mixed states, when the detail view is opened, then the number currently available is shown and the reserve action is offered only where appropriate.
FR18	The system SHOULD provide staff a keyboard command palette for jumping to a member or a title.	P2, Nice to Have	Given the staff interface, when Ctrl+K or Cmd+K is pressed, then a search overlay opens, is navigable by arrow keys, opens the highlighted result on Enter, and closes on Escape.

3.1.4 Circulation

ID	Requirement	Priority	Acceptance Criteria
FR19	The system MUST allow staff to check a copy out to a member as a single atomic operation.	P0, Must Have	Given an eligible member and an available copy, when checkout is processed, then a loan is created and the copy becomes CHECKED_OUT together, or neither change is written.
FR20	The system MUST set the due date from the member's own loan period.	P0, Must Have	Given a member with a 14 day loan period, when a book is issued, then the due date is 14 days from the issue date.
FR21	The system MUST refuse a checkout when the member is suspended, is at their borrowing limit, or has outstanding fines above the configured threshold, and MUST state the reason.	P0, Must Have	Given a member at their limit, when checkout is attempted, then it is refused and the specific reason is displayed at the desk before the book changes hands.
FR22	The system MUST process a return as a single atomic operation that closes the loan, computes any overdue fine, and either frees the copy or holds it for the reservation queue.	P0, Must Have	Given an overdue loan, when the copy is returned, then the loan is closed, a fine is raised for the days late, and the copy is either held for the next member in the queue or returned to the shelf.
FR23	The system MUST allow a loan to be renewed within a limit that varies by membership category.	P1, Should Have	Given a faculty member and a student who have each renewed twice, when both request a further renewal, then the faculty renewal succeeds and the student renewal is refused with the limit stated.
FR24	The system MUST refuse a renewal while another member is waiting for the title.	P1, Should Have	Given a pending reservation on a title, when the current borrower requests a renewal, then it is refused and the reason names the waiting queue.
FR25	The system MUST NOT issue a held copy to anyone other than the member it is being held for.	P0, Must Have	Given a copy in RESERVED state, when a different member presents it, then checkout is refused; when the member holding the reservation presents it, then checkout succeeds and the reservation is marked fulfilled.
FR26	The system MUST detect loans that have passed their due date without staff intervention.	P0, Must Have	Given active loans past their due date, when the scheduled sweep runs, then those loans are marked OVERDUE and their borrowers are notified.

3.1.5 Reservations

ID	Requirement	Priority	Acceptance Criteria
FR27	The system MUST allow a member to reserve a title, and MUST make the	P0, Must Have	Given a title with a copy on the shelf, when a member reserves it, then the

ID	Requirement	Priority	Acceptance Criteria
	hold immediately collectable when a copy is already free.		copy is held for that member and they are notified it is ready.
FR28	The system MUST place a member in an ordered queue when no copy is free, and MUST prevent duplicate active reservations on the same title.	P0, Must Have	Given all copies out, when a member reserves, then they receive a queue position; when the same member reserves the same title again, the request is refused with 409.
FR29	The system MUST promote the member at the front of the queue automatically when a copy is returned.	P0, Must Have	Given a queue on a title, when a copy of it is returned, then the front of the queue is marked ready, the copy stays held rather than returning to the shelf, and the member is notified.
FR30	The system MUST expire a ready hold that is not collected within the configured window and pass the copy to the next member.	P1, Should Have	Given a ready hold past its collection window, when the hourly job runs, then the hold is marked expired, the member is notified, and the copy is offered to the next member or returned to the shelf.
FR31	The system MUST allow a reservation to be cancelled by the member who made it or by staff, and MUST close the resulting gap in the queue.	P1, Should Have	Given a cancelled pending reservation, when the queue is read, then the members behind it have each moved up one position with no gap.

3.1.6 Fines and Enforcement

ID	Requirement	Priority	Acceptance Criteria
FR32	The system MUST calculate an overdue fine automatically on return as days late multiplied by the configured daily rate.	P0, Must Have	Given a loan returned four days late at the configured rate, when the return is processed, then a fine of four times that rate is raised against the member with the reason recorded.
FR33	The system MUST allow staff to record a fine as paid or to waive it, and MUST attribute both to the member of staff who did it.	P0, Must Have	Given an unpaid fine, when it is waived, then its status changes to WAIVED and an audit entry names the actor, the fine and the amount.
FR34	The system MUST block borrowing once a member's unpaid fines exceed the configured threshold.	P0, Must Have	Given unpaid fines above the threshold, when checkout is attempted, then it is refused and the outstanding amount and the limit are both stated.
FR35	The system MUST provide a defaulters report listing members with unpaid fines or overdue loans, with totals.	P1, Should Have	Given members in arrears, when the report is opened, then each is listed with their outstanding total, the number of unpaid fines and the number of overdue loans, ordered by amount owed.
FR36	The system MUST allow staff to suspend and reactivate a patron account.	P0, Must Have	Given a suspended member, when a checkout is attempted for them, then it is refused with suspension given as the

ID	Requirement	Priority	Acceptance Criteria
			reason, and the member is notified of the suspension.
FR37	The system MUST let a member see their own fines and their outstanding total.	P0, Must Have	Given a member with fines, when they open their fines view, then each fine is listed with its book, reason, amount and status, together with the total still owed.

3.1.7 Notifications and Scheduled Jobs

ID	Requirement	Priority	Acceptance Criteria
FR38	The system MUST record in-application notifications and expose an unread count.	P0, Must Have	Given a new notification, when the member signs in, then the unread indicator reflects it, and marking it read, individually or all at once, clears the count.
FR39	The system SHOULD dispatch the same notifications by email.	P1, Should Have	Given email transport configured, when a notifiable event occurs, then a message is sent and the dispatch outcome is recorded; when transport fails, the in-application notification is still delivered and circulation is unaffected.
FR40	The system MUST remind members of loans falling due within a configurable window.	P1, Should Have	Given a loan due inside the reminder window, when the daily job runs, then the borrower receives a due-soon notification naming the title and the date.
FR41	The system MUST notify borrowers when their loans become overdue.	P1, Should Have	Given loans past due, when the nightly sweep runs, then each borrower receives an overdue notification before the loans are marked.
FR42	The system MUST expire uncollected holds on a schedule without staff action.	P1, Should Have	Given ready holds past their window, when the hourly job runs, then they are expired and the copies are passed on.

3.1.8 Reporting, Audit and Oversight

ID	Requirement	Priority	Acceptance Criteria
FR43	The system MUST present staff a dashboard of active loans, overdue loans, total titles, copies by status, outstanding fines, member count and reservation activity.	P1, Should Have	Given circulation activity, when the dashboard is opened, then each figure matches the underlying records at the time of the request.
FR44	The system MUST report the most borrowed titles.	P1, Should Have	Given loan history, when the report is read, then titles are listed in descending order of total loans across all their copies.

ID	Requirement	Priority	Acceptance Criteria
FR45	The system SHOULD show a membership growth trend.	P2, Nice to Have	Given registrations over time, when the dashboard is opened, then new members per month for the last six months are charted.
FR46	The system MUST record an audit entry naming the actor for every action that moves stock, moves money, or changes an account.	P0, Must Have	Given a checkout, return, renewal, fine payment, fine waiver, suspension, reactivation or role change, when it completes, then an entry records the actor, the action, the entity and the time; and given an audit write failure, the underlying operation still succeeds.
FR47	The system MUST give administrators a per-person desk activity summary over a chosen window, counted by action.	P1, Should Have	Given staff activity, when an administrator opens the summary, then each staff member is listed with counts per action; and given a librarian's token, the request is refused with 403.
FR48	The system MUST give administrators a chronological audit trail.	P1, Should Have	Given recorded actions, when the trail is read, then entries appear newest first with the actor named.

3.1.9 System Configuration

ID	Requirement	Priority	Acceptance Criteria
FR49	The system MUST hold library policy as editable settings rather than as values fixed in code, covering at least loan period, daily fine rate, renewal allowances, borrowing limits, the fine block threshold, the hold collection window and the reminder window.	P0, Must Have	Given a changed setting, when the next transaction runs, then the new value is applied without a redeployment.
FR50	The system MUST restrict editing settings to administrators while allowing librarians to read them.	P0, Must Have	Given a librarian, when they read settings the request succeeds, and when they attempt to change one it is refused with 403.
FR51	The system SHOULD seed default settings, a subject taxonomy and demonstration accounts repeatably.	P1, Should Have	Given the seed is run twice, when it completes, then no duplicate records are created and existing data is preserved.

3.2 Non-Functional Requirements

ID	Category	Requirement	Metric or Target
NFR01	Performance	Catalogue search and list endpoints MUST respond within 500 ms at the 95th percentile for a catalogue of 10,000 titles, using pagination and indexed columns.	Under 500 ms P95
NFR02	Availability	The service MUST remain available through semester peak periods.	99.5 percent monthly

ID	Category	Requirement	Metric or Target
NFR03	Security	Passwords MUST never be stored or logged in plain text, and MUST be hashed with bcrypt at cost 10 or higher. API access MUST use signed tokens with a short access lifetime.	Security review pass
NFR04	Data Integrity	Any operation that changes stock state and financial state together MUST run inside a single database transaction.	No partial writes under fault injection
NFR05	Authorisation	Every access rule MUST be enforced server-side. The interface MUST NOT be the only place a restriction exists.	Direct API calls obey all role rules
NFR06	Compliance	The system MUST comply with Ghana's Data Protection Act, 2012 (Act 843). Personal data MUST be limited to what the service needs, and any required field MUST state its purpose at the point of collection.	Legal review pass
NFR07	Auditability	Every action that moves stock, moves money or changes an account MUST be attributable to an identified actor.	100 percent of such actions attributed
NFR08	Usability	A trained member of staff MUST be able to complete a checkout using a barcode scanner without touching the mouse.	Two scans and two keystrokes
NFR09	Responsiveness	Member-facing screens MUST be usable on a phone with no horizontal page scrolling.	Usable at 390 px viewport width
NFR10	Accessibility	Interactive controls MUST be reachable by keyboard, MUST carry accessible names, and text MUST remain legible in both light and dark themes.	Contrast and keyboard audit pass
NFR11	Reliability	A failure in a secondary concern such as email or audit logging MUST NOT prevent a circulation transaction from completing.	Circulation succeeds with mail and audit disabled
NFR12	Maintainability	Each feature module MUST follow the same layering, and the automated test suite MUST pass before any change is merged.	Suite green on every merge

4. System Design

4.1 High-Level Architecture

ALS is a client-server web application. A React single-page application communicates with a Node and Express REST API over HTTPS, and the API is the only component that talks to the PostgreSQL database. Every request that is not public carries a JWT bearer token, from which the server derives the caller's identity and role before any handler runs. The two applications live in one npm workspace repository and are developed together, so a change to an API contract and the screen that consumes it travel in the same commit.

Component	Responsibility	Key Technology
Web Client	Member portal and staff back office, routing, forms, role-aware navigation, light and dark themes, responsive layouts down to phone width.	React 18, Vite, TypeScript, Tailwind CSS
Client Data Layer	Server state caching, request de-duplication, background refresh and cache invalidation after mutations.	TanStack Query, Axios
API Server	REST endpoints, authentication, role checks, request validation, business rules, transaction boundaries.	Node 20, Express 4, TypeScript
Request Validation	Schema validation of every request body and query string before a handler executes, with typed output.	Zod
Data Access	Typed queries, relations, migrations and interactive transactions.	Prisma 6
Database	Relational store for users, catalogue, copies, loans, reservations, fines, notifications, settings and the audit log.	PostgreSQL 15
Authentication	Password hashing, access and refresh token issue and verification, role hierarchy.	bcryptjs, jsonwebtoken (HS256)
Notifications	In-application notification records plus outbound email dispatch.	PostgreSQL, Nodemailer over SMTP
Scheduler	Overdue sweep, due-soon reminders and hold expiry, toggleable by environment.	node-cron
Reporting	Aggregate queries for the dashboard, borrowing rankings, stock health and membership trend.	Prisma aggregation, Recharts
External Metadata	ISBN lookup for book creation and cover artwork for the catalogue, used optionally.	Open Library API
Testing	Integration tests exercising real HTTP routes against a real database.	Vitest, Supertest

4.2 Low-Level Design

The backend is one Express application composed of feature modules. Every module follows the same layering: routes declare the path and attach middleware, a controller reads the request and shapes the response, a service holds the business rules, and Prisma performs the data access. Controllers stay thin deliberately, so that a rule has exactly one home and can be tested without an HTTP request.

Two modules sometimes need to affect each other, for example a return has to promote the reservation queue. Importing one service into the other in both directions would create a cycle, so those effects are registered as hooks at start-up instead: circulation exposes hook points,

and the reservation module fills them in. Circulation therefore knows that something may claim a returned copy without knowing what.

- **Checkout flow:** staff scan the member, then the copy. Inside one transaction the system evaluates eligibility (suspension, borrowing limit, fine threshold), resolves the copy either from the scanned accession number or from the first free copy of the title, verifies that a held copy belongs to this member, flips the copy to CHECKED_OUT, marks any fulfilled reservation, and creates the loan with a due date derived from the member's loan period. The controller then records an audit entry naming the member of staff.
- **Return flow:** staff scan the copy alone, because the accession number identifies the loan. Inside one transaction the system finds the open loan, compares the return date with the due date and raises a fine for the days late at the configured rate, closes the loan, then offers the copy to the reservation queue. If someone is waiting, the copy stays RESERVED and is held for them; if not, it becomes AVAILABLE. Notifications raised during the transaction are queued and dispatched only after it commits, so a mail failure can never roll back a return.
- **Reservation flow:** reserving a title with a free copy holds that copy immediately and marks the reservation READY with an expiry. Reserving a title with nothing free appends the member to a PENDING queue by position. Promotion happens on return and on cancellation of a ready hold. Expiry runs hourly and passes the copy on. Cancelling a pending reservation re-sequences everyone behind it so the queue never develops a gap.
- **Renewal flow:** renewal reads the per-category allowance first, falling back to a global default where no category-specific value is configured, so faculty and undergraduates are not held to the same limit. It is refused outright while any member is waiting for the title, because extending one person's loan at the expense of a queue is the behaviour the queue exists to prevent.
- **Scheduling flow:** three cron jobs run inside the API process. Nightly, borrowers of loans past due are notified and those loans are marked OVERDUE. Daily, members with loans falling due inside the reminder window are reminded. Hourly, ready holds past their collection window are expired and their copies passed on. All three are pure functions over the database and can be invoked directly by a test without waiting for a schedule.
- **Audit and oversight flow:** checkout, return, renewal, fine payment, fine waiver, suspension, reactivation and role change each write an audit entry carrying the actor, the action, the entity, the entity identifier and a small metadata document. Recording is done at the controller layer, where the authenticated user is available, so service signatures stay free of request concerns. The audit write never throws: losing a log line is bad, but refusing to lend a book because a log line could not be written is worse, so failures are reported to standard error and the transaction stands.
- **Authorisation model:** roles are ranked MEMBER 1, LIBRARIAN 2, ADMIN 3. Route middleware requires a minimum rank. Account actions apply a second, separate test comparing actor and target, described in section 2.1.3. Members are additionally scoped to their own records inside the services themselves, so a member requesting another member's loan receives a not-found rather than a record they should not see.

- **Data protection:** passwords are stored only as bcrypt hashes. Tokens are signed with HS256 and access tokens are short lived, with refresh handled separately. Personal data is limited to what the library service needs: name, email, member identifier and phone number. The phone number is required because reminders and fine notices depend on being able to reach a borrower, and the registration form states that purpose beside the field, which is what Act 843 asks of a required field. Users, books and copies are soft deleted so that history remains interpretable after a record is withdrawn.

4.3 Data Model

One PostgreSQL schema holds fourteen entities. Primary keys are UUIDs. Records carry creation and update timestamps, and the entities whose history matters carry a soft-delete marker rather than being physically removed.

Entity	Purpose	Key Fields and Relations
User	Every account, whatever the role.	fullName, email (unique), identifier (unique, alternate login), phone, passwordHash, role, membershipType, status, borrowingLimit, loanPeriodDays, soft delete. Has loans, reservations, fines, notifications.
Author	A writer, shared across titles.	name (unique). Many-to-many with Book.
Publisher	A publishing house.	name (unique), address. One-to-many with Book.
Category	A subject in the taxonomy.	name (unique), description. One-to-many with Book.
Book	A bibliographic title, not a physical object.	isbn (unique), title, description, publicationYear, edition, language, coverImageUrl, soft delete. Belongs to a Category and a Publisher, has Authors, Copies and Reservations.
BookCopy	One physical volume on a shelf.	accessionNumber (unique), shelfLocation, status, acquiredDate, soft delete. Belongs to a Book, has Loans.
Loan	One issue of one copy to one member.	checkoutDate, dueDate, returnDate, status, renewalCount. Belongs to a BookCopy and a User, has at most one Fine.
Reservation	A member's claim on a title.	reservationDate, status, queuePosition, readyAt, expiresAt. Belongs to a Book and a User.
Fine	Money owed for an overdue return.	amount (decimal 10,2), reason, status, paidAt. Belongs to one Loan and one User.
Notification	One message to one member.	type, channel (in-app or email), title, message, read, emailSent. Belongs to a User.
Setting	One item of library policy.	key (unique), value, description. Read by services at the point of use.
AuditLog	One recorded action.	actorId, action, entity, entityId, metadata, createdAt. Indexed by entity and by actor.

4.4 Testing and Verification

Backend tests run with Vitest and drive the application over real HTTP using Supertest against a real PostgreSQL database rather than mocks, because the behaviour most worth protecting here is transactional and a mock cannot demonstrate that a transaction rolled back. Each suite creates and cleans up its own data. The rules under test are the ones that cost the library something when they break: a copy issued twice, a fine calculated wrongly, a queue jumped, or a librarian suspending an administrator.

Suite	Tests	What It Protects
Health	1	Process is up and the database is reachable.
Authentication	8	Registration, sign-in by email and by member ID, token refresh, rejection of bad credentials.
Phone validation	5	Ghanaian number formats accepted, missing and malformed numbers refused, and the refusal explains itself.
Authorisation	6	The actor-versus-target rule: no acting on a superior, a peer, or oneself.
Catalogue	5	Book creation, duplicate ISBN refusal, search and filtering.
Accession numbering	5	Sequential numbering preserves zero padding and refuses the whole batch on a collision.
Circulation	4	Transactional checkout and return, due date derivation, overdue fine calculation.
Eligibility	3	Suspension, borrowing limit and fine threshold each block a checkout.
Reservations	6	Queue ordering, promotion on return, expiry, cancellation and re-sequencing.
Fines	5	Fine creation, payment, waiver and the defaulters report.
Notifications and scheduler	4	Notification records, the overdue sweep and the reminder window.
Total	52	11 suites, all passing.

4.5 Delivery Phases

The system was built in phases, each ending in a verification gate: the phase was not considered finished until its behaviour had been demonstrated against a running system.

Phase	Scope	Status
Phase 0	Scaffold: both applications boot, Prisma connects to PostgreSQL, health check responds.	Complete
Phase 1	Authentication and role-based access control: registration, sign-in, refresh, hashing, role middleware.	Complete
Phase 2	Catalogue and inventory: book records, physical copies, shared lookups.	Complete
Phase 3	Search: public catalogue, filters, book detail with live availability.	Complete
Phase 4	Circulation: transactional checkout, return and renewal, overdue sweep.	Complete

Phase	Scope	Status
Phase 5	Reservations: ordered queue, promotion on return, cancellation and expiry.	Complete
Phase 6	Fines and enforcement: automatic fines, payment and waiver, defaulters, suspension.	Complete
Phase 7	Notifications and reporting: in-application and email, scheduled jobs, dashboard charts.	Complete
Phase 8	Polish: responsive navigation, error boundary, empty and loading states, tests, documentation.	Complete
Phase 9	Institutional identity, theming and oversight: KNUST branding, light and dark themes, shared icon set, barcode workflow, audit trail and staff activity.	Complete

5. Revenue and Sustainability Model

ALS is institutional software, not a consumer product, so it is not sold to the students who use it. Students pay nothing. Sustainability comes from three directions: revenue the library already collects but currently collects inconsistently, cost the institution avoids by not licensing a commercial system, and the possibility of deploying the same system at other Ghanaian institutions.

Stream	Description	Basis
Overdue fines	Fines already form part of library policy. The change ALS makes is that they are computed uniformly from a configured rate rather than by hand, are attached to a named loan, and cannot be written off without a record of who approved it.	Configured daily rate, seeded at GH₵0.50 per overdue day and editable by an administrator
Cost avoidance	The institution does not pay a recurring licence for a commercial integrated library system, and does not pay for the consultancy that a large open source deployment typically requires.	Recurring subscription and implementation cost not incurred
Institutional funding	Hosting, backup and maintenance are funded from the existing university ICT budget, in the same way as other internal services.	Internal budget line
Lost and damaged item charges	Copies already carry LOST and DAMAGED states. Attaching a replacement charge to those states is a small extension of the existing fine model.	Replacement cost per title, set by library policy
Institutional licensing	The same system can be deployed for other Ghanaian universities, technical universities, colleges of education and senior high schools, whose libraries face the same constraints.	Per-deployment fee plus optional annual support
Departmental deployments	Faculty and departmental libraries within KNUST can run their own instance or their own branch on a shared instance.	Internal transfer or shared hosting

Note on fines as revenue. Fine income is not the point of the fine. Its purpose is to return books to circulation for the next borrower, and a library that started optimising for fine income would be working against its own members. ALS therefore treats fines as an enforcement mechanism with a financial record attached, which is why waiving one is a first-class, fully audited action rather than something to be discouraged.

6. Payment Concept

6.1 Current Flow: Settlement at the Desk

In the current build, money changes hands the way it already does at the library: in person, at the circulation desk, under existing library cash-handling procedure. What ALS adds is the record. A member of staff records the settlement against the specific fine, and the system attributes it to them. The system tracks the obligation and its discharge; it does not hold funds and is not a payment processor.

Step	Actor	Action
1	System	Raises a fine automatically when an overdue copy is returned, computed as days late multiplied by the configured daily rate, and notifies the member.
2	Member	Sees the fine and the outstanding total in their own fines view, with the title and the reason it was raised.
3	System	Blocks further borrowing once the unpaid total exceeds the configured threshold, stating the amount and the limit at the desk.
4	Member	Presents payment at the circulation desk under existing library procedure.
5	Librarian	Records the fine as paid, or waives it where library policy allows.
6	System	Marks the fine PAID or WAIVED, writes an audit entry naming the member of staff, the fine and the amount, and lifts the borrowing block once the outstanding total falls below the threshold.

6.2 Planned Flow: Mobile Money Settlement

The desk flow keeps a student in a queue to pay a small sum, which is the part worth removing. The planned extension lets a member clear a fine from their phone using the payment method they already use daily. Mobile money (MTN MoMo, Telecel Cash and AirtelTigo Money) would be integrated through a Ghanaian aggregator such as Paystack or Hubtel.

Step	Actor	Action
1	Member	Opens their fines view and chooses to pay, either a single fine or the outstanding total.
2	Payment gateway	Presents a mobile money prompt for the member's registered number.
3	Member	Authorises the payment with their mobile money PIN.
4	Payment gateway	Confirms the transaction and calls back to the ALS server with the result.

Step	Actor	Action
5	System	Verifies the callback signature, marks the fine PAID with the transaction reference recorded, and writes an audit entry with the gateway as the actor.
6	System	Lifts the borrowing block automatically once the outstanding total falls below the threshold, and notifies the member.

Two constraints shape this design. The fine record remains the single source of truth, so a gateway outage never leaves a member unable to settle: the desk flow stays available in parallel and both routes write the same record. And a payment is only ever recognised from a verified server-to-server callback, never from the browser reporting success, because a client-reported payment is a client-controlled payment.

7. API Endpoints

All endpoints are served by the Express application under the /api prefix. Only the health check and the three authentication routes are public; every other route requires an Authorization header carrying a bearer access token. In the Access column, Member means any signed-in user acting on their own records, Staff means LIBRARIAN or above, and Admin means ADMIN only.

7.1 Health and Authentication

Method	Endpoint	Access	Description
GET	/api/health	Public	Liveness check that also verifies database connectivity.
POST	/api/auth/register	Public	Create a member account with name, email, password and phone number.
POST	/api/auth/login	Public	Authenticate with email or member ID plus password, returning access and refresh tokens.
POST	/api/auth/refresh	Public	Exchange a valid refresh token for a new access token.

7.2 Users and Membership

Method	Endpoint	Access	Description
GET	/api/users/me	Member	Read the signed-in user's own profile.
PATCH	/api/users/me	Member	Update the signed-in user's own profile details.
POST	/api/users/me/change-password	Member	Change own password, requiring the current one.
GET	/api/users	Staff	List and search accounts, filterable by role and status, paginated.

Method	Endpoint	Access	Description
PATCH	/api/users/:id/status	Staff	Suspend or reactivate an account, subject to the actor-versus-target rule.
PATCH	/api/users/:id/role	Admin	Change an account's role.
PATCH	/api/users/:id/membership	Admin	Change an account's membership category and the limits that follow from it.

7.3 Catalogue and Inventory

Method	Endpoint	Access	Description
GET	/api/catalog/books	Member	Search and filter the catalogue by title, author, ISBN, category and availability, paginated.
GET	/api/catalog/books/:id	Member	Read one title with its authors, publisher, category and live copy availability.
POST	/api/catalog/books	Staff	Create a title.
PUT	/api/catalog/books/:id	Staff	Update a title.
DELETE	/api/catalog/books/:id	Staff	Soft-delete a title.
GET	/api/catalog/categories	Member	List subject categories.
GET	/api/catalog/publishers	Member	List publishers.
GET	/api/catalog/authors	Member	List authors.
POST	/api/catalog/categories	Staff	Create a subject category.
POST	/api/catalog/publishers	Staff	Create a publisher.
POST	/api/catalog/authors	Staff	Create an author.
GET	/api/catalog/books/:id/copies	Staff	List the physical copies of a title.
POST	/api/catalog/books/:id/copies	Staff	Add one or several copies with sequential accession numbers.
GET	/api/catalog/copies/lookup	Staff	Find a copy by accession number, the endpoint the barcode scanner drives.
PATCH	/api/catalog/copies/:copyId/status	Staff	Set a copy's status, for example to LOST or DAMAGED.
DELETE	/api/catalog/copies/:copyId	Staff	Soft-delete a copy.

7.4 Circulation

Method	Endpoint	Access	Description
GET	/api/circulation/my-loans	Member	List the signed-in member's own loans, optionally active only.

Method	Endpoint	Access	Description
POST	/api/circulation/loans/:id/renew	Member	Renew a loan. Members may renew their own; staff may renew any.
POST	/api/circulation/checkout	Staff	Issue a copy to a member in one transaction, after evaluating eligibility.
POST	/api/circulation/return	Staff	Take a copy back in one transaction, raising any overdue fine and releasing or holding the copy.
GET	/api/circulation/loans	Staff	List all loans, filterable by status and member, paginated.
GET	/api/circulation/eligibility/:userId	Staff	Report whether a member may borrow, with the reasons if not.

7.5 Reservations

Method	Endpoint	Access	Description
POST	/api/reservations	Member	Reserve a title, going straight to ready if a copy is free.
GET	/api/reservations/mine	Member	List the signed-in member's own reservations.
POST	/api/reservations/:id/cancel	Member	Cancel a reservation. Members may cancel their own; staff may cancel any.
GET	/api/reservations	Staff	Read the reservation queue, filterable by title and status.

7.6 Fines

Method	Endpoint	Access	Description
GET	/api/fines/mine	Member	List the signed-in member's own fines with their outstanding total.
GET	/api/fines	Staff	List all fines, filterable by status and member, paginated.
GET	/api/fines/defaulters	Staff	List members with unpaid fines or overdue loans, with totals.
POST	/api/fines/:id/pay	Staff	Record a fine as paid, attributed to the member of staff.
POST	/api/fines/:id/waive	Staff	Waive a fine, attributed to the member of staff.

7.7 Notifications, Reporting and Settings

Method	Endpoint	Access	Description
GET	/api/notifications	Member	List the signed-in member's in-application notifications with the unread count.
POST	/api/notifications/:id/read	Member	Mark one notification as read.
POST	/api/notifications/read-all	Member	Mark all notifications as read.
GET	/api/reports/dashboard	Staff	Live figures: active and overdue loans, titles, copies by status, outstanding fines, members, reservations, membership trend.
GET	/api/reports/most-borrowed	Staff	Titles ranked by total loans across all their copies.
GET	/api/reports/stock-status	Staff	Copy counts grouped by status.
GET	/api/reports/staff-activity	Admin	Per-person desk activity over a window, counted by action.
GET	/api/reports/audit-log	Admin	Recent audit entries, newest first, with the actor named.
GET	/api/settings	Staff	Read library policy settings.
PATCH	/api/settings/:key	Admin	Change one library policy setting.

Response conventions. A successful request returns the resource or collection directly as JSON. A failure returns a single consistent envelope, { "error": { "code", "message", "details" } }, so that the client has exactly one shape to handle. Validation failures return 422 with the offending fields, authentication failures 401, authorisation failures 403, unique-constraint violations 409, and missing records 404. Stack traces are included only in development.

8. Full Stack

Layer	Technology	Why This Choice
Web Client	React 18 with TypeScript	Component model suits an interface with two distinct role-based shells over one shared component library. TypeScript catches contract drift between client and API at compile time.
Build Tool	Vite 6	Near-instant development server start and hot module replacement, with a proxy that forwards /api to the backend so no CORS configuration is needed in development.
Styling	Tailwind CSS 3	Utility classes with semantic colour tokens defined as CSS variables, which is what makes a genuine light and dark theme achievable without maintaining two stylesheets.
Client Routing	React Router 6	Nested routes let a role-aware layout and a route guard wrap whole branches of the application rather than being repeated on each screen.

Layer	Technology	Why This Choice
Server State	TanStack Query with Axios	Caching, de-duplication and invalidation after mutations, so a checkout refreshes the eligibility, loan list and activity views without manual coordination.
Charts	Recharts	Declarative React charting for the dashboard, avoiding an imperative charting library inside a React tree.
API Framework	Node 20 with Express 4	Mature, well understood middleware model. Sharing one language across client and server keeps types and validation logic consistent.
Validation	Zod	One schema validates a request and produces its TypeScript type, so a validated request cannot drift from its declared shape.
ORM	Prisma 6	Type-safe queries, first-class interactive transactions for the checkout and return paths, and versioned migrations.
Database	PostgreSQL 15	ACID guarantees, which the circulation model depends on absolutely, plus exact decimal arithmetic for fines and JSON columns for audit metadata.
Authentication	jsonwebtoken with bcryptjs	Stateless HS256 tokens carrying identity and role, so authorisation needs no session store. bcrypt at cost 10 or higher for password storage.
Email	Nodemailer over SMTP	Standard transport that works with an institutional relay in production and falls back to a console or Ethereal preview in development.
Scheduling	node-cron	In-process scheduling with no additional infrastructure, and each job is a plain function a test can call directly.
Testing	Vitest with Supertest	Integration tests over real HTTP against a real database, which is the only way to demonstrate that a transaction actually rolled back.
Repository	npm workspaces	Client and server versioned together, with one command to install and one to run both applications.

9. Future Enhancements

9.1 Known Gaps

Two things are worth stating plainly rather than leaving to be discovered:

- **Currency formatting is inconsistent between layers.** The client renders every amount in Ghana cedis, but a handful of server-generated message strings, specifically the fine notification and the eligibility refusal reason, still format amounts with a dollar sign. The stored values are correct and unaffected; only those message strings need to be brought onto the cedi. This should be fixed before any pilot deployment.
- **Notifications are in-application and email only.** Phone numbers are now collected and required, and the stated purpose of collecting them is reminders and fine notices, but SMS

dispatch is not yet implemented. Until it is, the phone number supports staff contacting a borrower directly rather than automated messaging.

9.2 Planned Enhancements

- **Mobile money fine settlement:** implement the flow described in section 6.2 so a member can clear a fine from their phone through MTN MoMo, Telecel Cash or AirtelTigo Money, with the borrowing block lifting automatically on a verified callback.
- **SMS notifications:** dispatch due-soon reminders, overdue notices and reservation-ready alerts by SMS through a Ghanaian gateway, which reaches a student who is not checking email far more reliably than mail does.
- **University single sign-on:** authenticate students and staff against existing university credentials so that no separate library password exists, and so that membership category and status follow enrolment automatically rather than being maintained by hand.
- **Library standards interoperability:** MARC 21 record import and export, Z39.50 for federated search across institutions, and SIP2 for self-service kiosks. These are what would let ALS exchange records with other Ghanaian academic libraries instead of being an island.
- **RFID and self-service:** extend the accession-number scanning model to RFID tags, enabling self-service borrowing stations and rapid shelf inventory, which is the single largest saving available to a library of this size.
- **Multi-branch and departmental libraries:** model holdings by branch so that faculty and departmental collections can be searched together while remaining separately administered, with transfers between branches tracked.
- **Recommendations and reading lists:** surface related titles from borrowing patterns and let lecturers publish a course reading list that links directly to catalogue availability, turning a reading list into a reservation in one action.
- **Native mobile application with offline access:** a mobile client that caches a member's loans, due dates and reservation positions, so the information a student most often needs remains available on an intermittent campus connection.
- **Fine appeals workflow:** let a member contest a fine in the application and route it to a librarian for a decision, so that the reasoning behind a waiver is recorded alongside the waiver itself.

End of Document